SE 200 / SE 201 · LIVE C & DATA-STRUCTURES

SE-ViZ — visual memory, locked.

An IDE-grade teaching surface that runs C visually. The first six layouts prove the live-memory vocabulary; the last three extend the same color and type system to *concept diagrams* — build pipeline, preprocessor, and dual-trace debugging — where dashed neutral boxes read as process, not memory. Dark is primary; flip the toggle to verify the light variant.

FOUNDATION

Built on the warm-neutral Temar base — `JetBrains Mono` for everything technical, Manrope for chrome, hairline 1px borders, 8–12px radii, flat surfaces, no glow.

COLOR = MEANING

Three hues only: `pointer` azure, `heap` teal, stack neutral — plus state flags for active, value-changed, and the four allocation states. Never decorative.

ONE THING IS CURRENT

Exactly one element per step carries the amber active ring + badge. Hover any cell or frame for its full type, value and address.

01 Recursion / call stack — stacked frames, each with its own n, deepest frame active

SE-ViZ | SE 201 › Recursion › factorial.c

Step 7 / 14

FACTORIAL.C

```c
#include <stdio.h>

int factorial(int n) {
    if (n <= 1)
        return 1;
    return n * factorial(n - 1);
}

int main(void) {
    int result = factorial(4);
    printf("%d\n", result);
    return 0;
}
```

CALL STACK — EACH CALL NESTS ONE LEVEL DEEPER · NEWEST FRAME AT THE BOTTOM

main() depth 0 · @0x7ffee570
- result int ?

factorial(n=4) depth 1 · @0x7ffee540
- n int 4
- return int 4 * factorial(3)

factorial(n=3) depth 2 · @0x7ffee510
- n int 3
- return int 3 * factorial(2)

factorial(n=2) depth 3 · @0x7ffee4e0
- n int 2
- retur

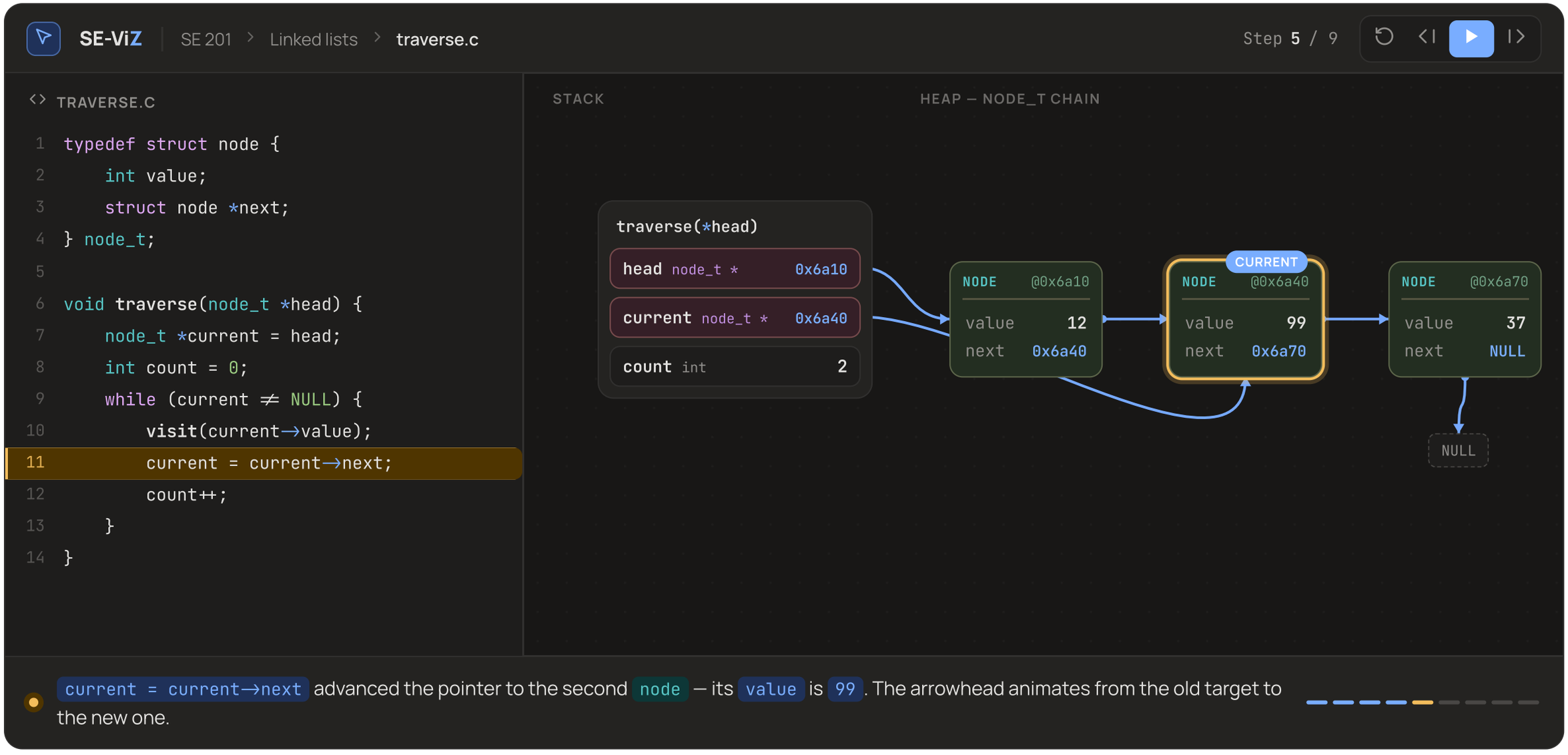
ACTIVE

factorial(n=1) depth 4 · @0x7ffee4b0
- n int 1
- return int 1

Base case reached — `factorial(1)` returns `1`. Frames now unwind from the bottom up, each multiplying by its own `n`.

Motion: each frame springs in on call (~250 ms, physical not bouncy) and springs out on return. Stepping back reverses the same transition. The base-case return value pulses once (green), then fades.

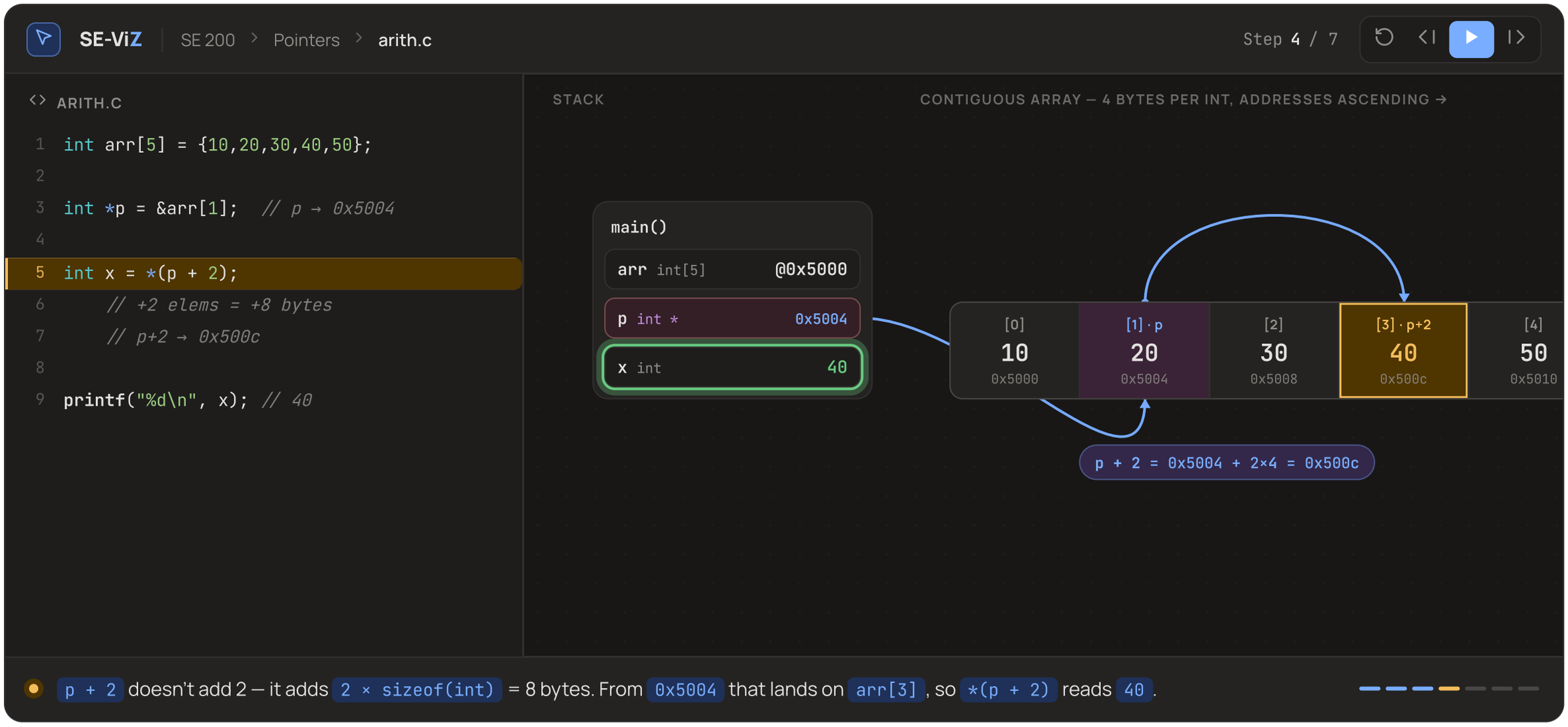
02 Linked-list traversal — head & current pointers walking a heap-allocated chain



`current = current→next` advanced the pointer to the second `node` — its `value` is `99`. The arrowhead animates from the old target to the new one.

Motion: on reassignment the `current` arrowhead glides from node 0 to node 1 (~250 ms). Hovering any node reveals its full type, value and address; hovering the frame lists its locals.

03 Pointer arithmetic — `*(p + 2)` over a contiguous array, scaled by `sizeof(int)`



Motion: the p+2 arc draws across two cells while the byte-offset annotation counts up (4 → 8); the destination cell lights amber as x receives **40**. The shared cell borders make "contiguous" literal — no gaps between elements.

04 malloc / free — the heap

— the four allocation states made visually distinct: allocated, freed, dangling, leaked

SE-ViZ | SE 200 › Dynamic memory › heap.c

Step 9 / 16

HEAP.C

```
1  char *buf = malloc(32);
2  strcpy(buf, "hello");
3
4  char *old = malloc(32);
5  free(old);   // old now dangles
6
7  char *tmp = malloc(32);
8  free(tmp); tmp = NULL;
9
10 char *c = malloc(32);
11 c = NULL;  // 0x9060 leaked
```

STACK

demo()

buf char * 0x9000

old char * 0x9040

HEAP — ADDRESSES ASCENDING ↓

ALLOCATED
32 BYTES @0x9000
content "hello\0"

FREED
32 BYTES @0x9020
content — returned —

DANGLING
32 BYTES @0x9040
content — freed —

LEAKED
32 BYTES @0x9060
content 0x?? · unreachable

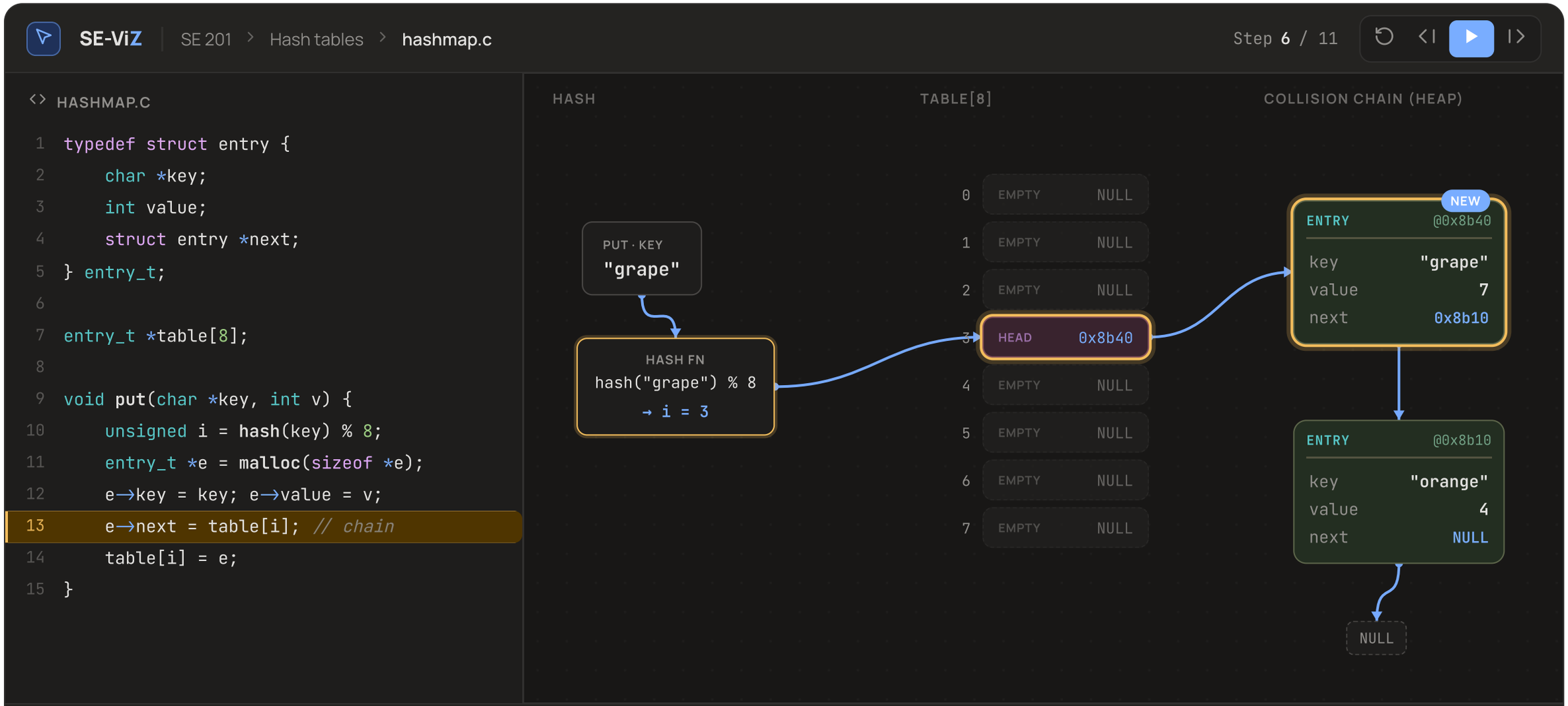
OUTPUT · HEAP WATCH

```
$ ./demo
⚠ dangling: read through old → 0x9040 was freed at heap.c:5
⚠ leak: 32 bytes @ 0x9060 allocated at heap.c:10, never freed
```

`free(old)` released the block at `0x9040` — but `old` still holds that address. It is now a `dangling pointer`; reading `*old` is undefined behaviour.

Motion: on `free` the block desaturates and the hatch fades in; the surviving pointer's arrow recolors to the danger hue. Leaked blocks keep their teal fill but gain the warning marker — no incoming arrow makes the leak self-evident.

05 Hash table — a key hashed to a bucket index, with a collision chain hanging off one bucket



`hash("grape")` lands on `bucket 3` — already holding `"orange"`. Instead of overwriting, the new entry is prepended and its `next` points at the old head, forming a collision chain.

Motion: the key flows into the hash box, the resulting bucket pulses amber, then the new entry slides in and re-points `table[3]` at itself while its `next` latches onto the previous head — the chain grows without disturbing the existing entry.

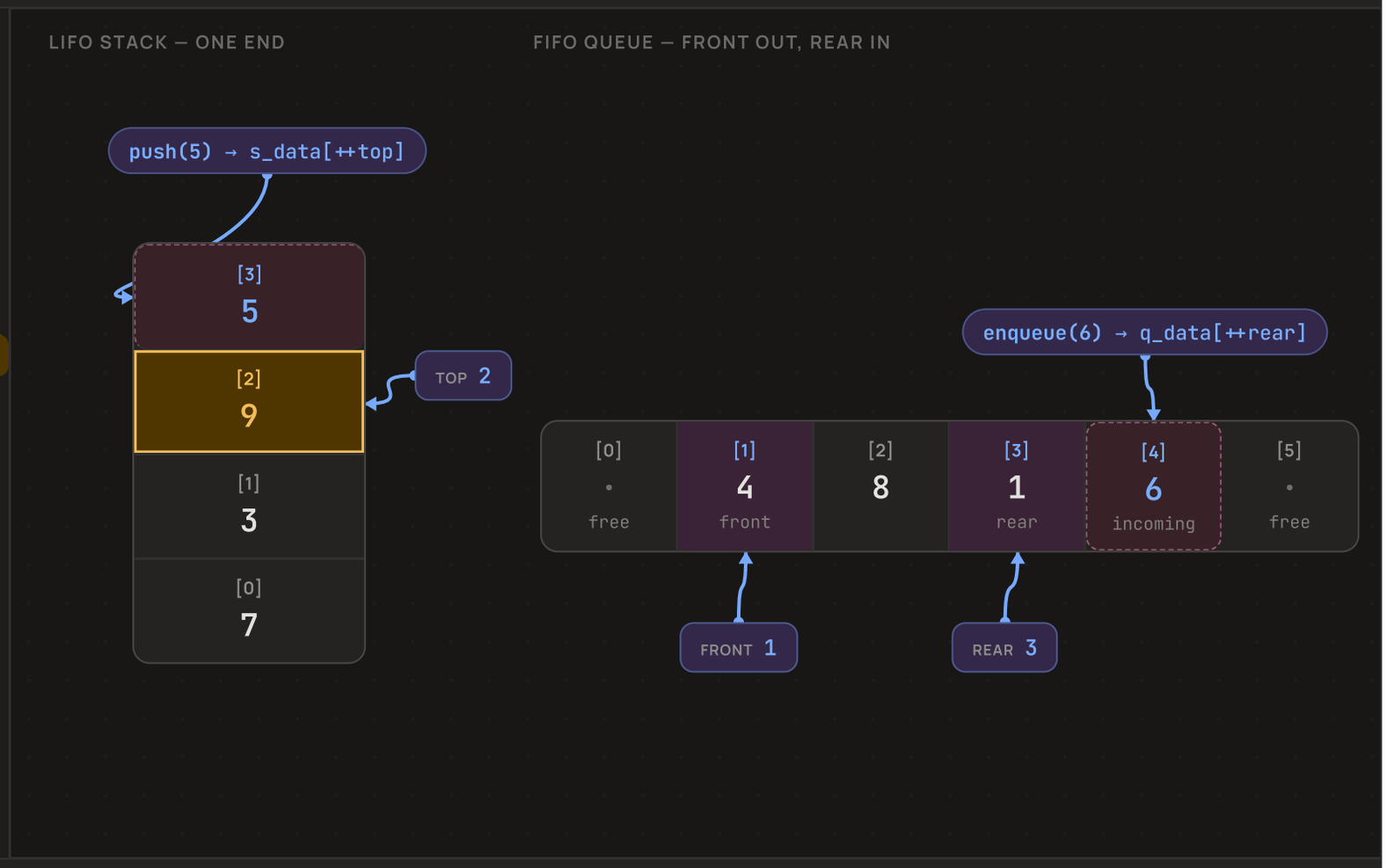
06 Stack & queue — LIFO grows at one end; FIFO uses two cursors. Shown mid push & mid enqueue

SE-ViZ | SE 201 › Linear ADTs › adt.c

Step 3 / 8

ADT.C

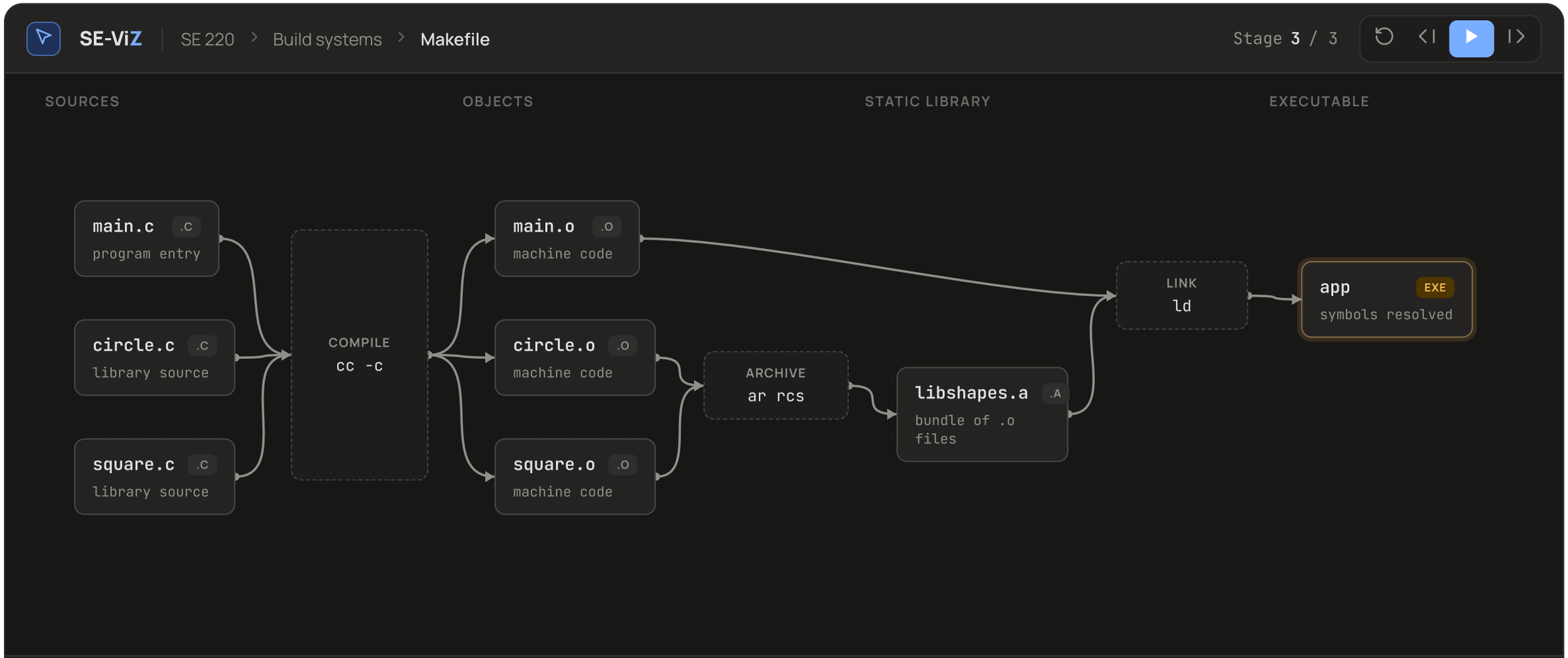
```
1  // array-backed ADTs
2  int s_data[6]; int top = 2;
3  int q_data[6];
4  int front = 1, rear = 3;
5
6  void push(int x) {
7      s_data[++top] = x; // LIFO
8  }
9  int pop(void) {
10     return s_data[top--];
11 }
12
13 void enqueue(int x) {
14     q_data[++rear] = x; // FIFO
15 }
16 int dequeue(void) {
17     return q_data[front++];
18 }
```



A **stack** touches only `top` — `push` and `pop` both work the same end (LIFO). A **queue** keeps two cursors: `enqueue` appends at `rear`, `dequeue` removes at `front` — first in, first out.

Motion: the pushed value drops into the stack's next slot as `top` increments to amber; on the queue, the incoming value slides into `rear+1` while `front` holds — the two cursors moving independently is the whole point of FIFO.

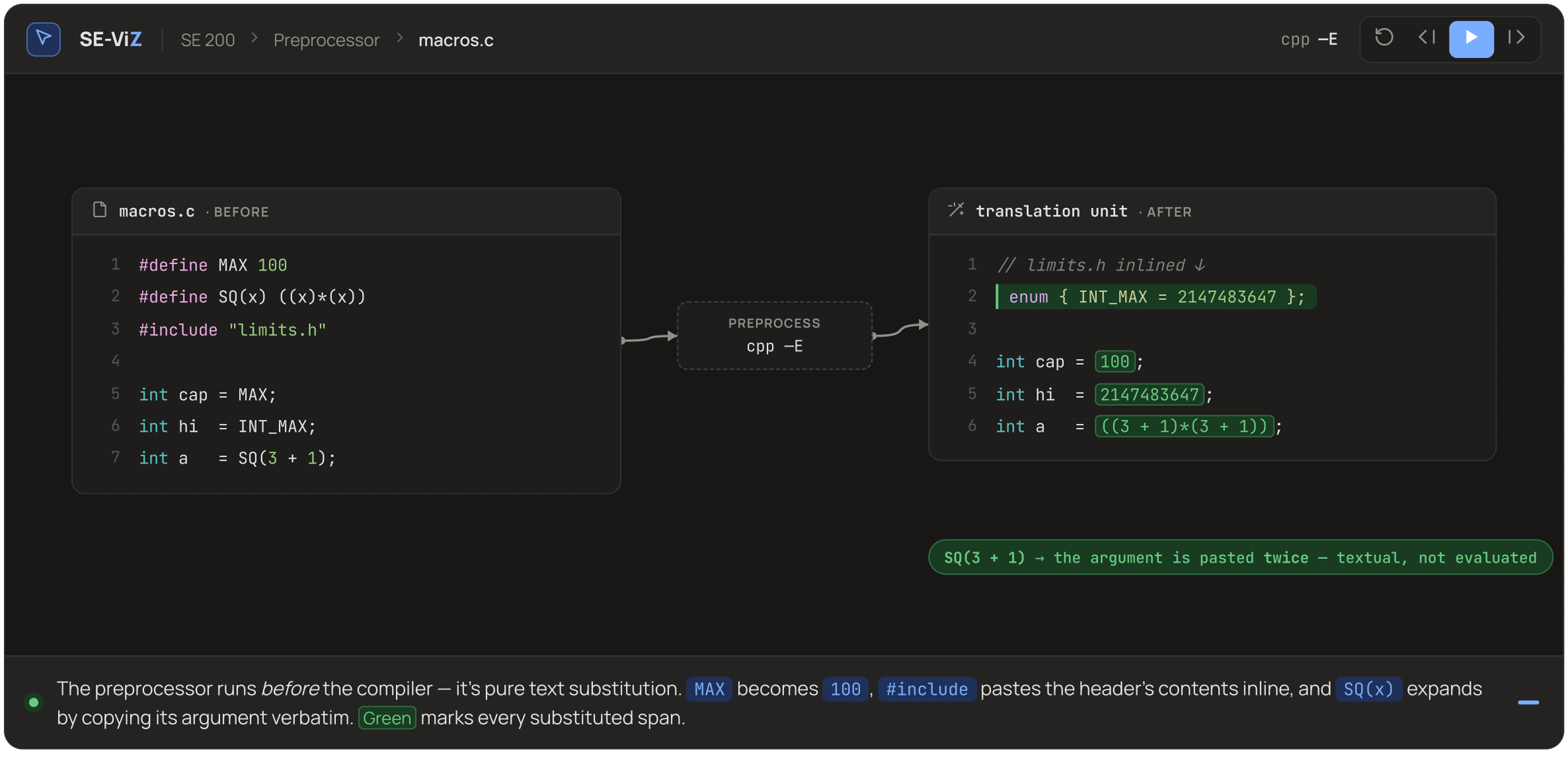
07 Static-library build pipeline — `.c → .o → .a → binary`. Dashed boxes are *processes*, solid cards are *artifacts*



Each `.c` compiles independently to a `.o` of machine code. `ar` bundles the library objects into a `.a` archive; the linker then resolves `main.o`'s symbols against that archive into one executable.

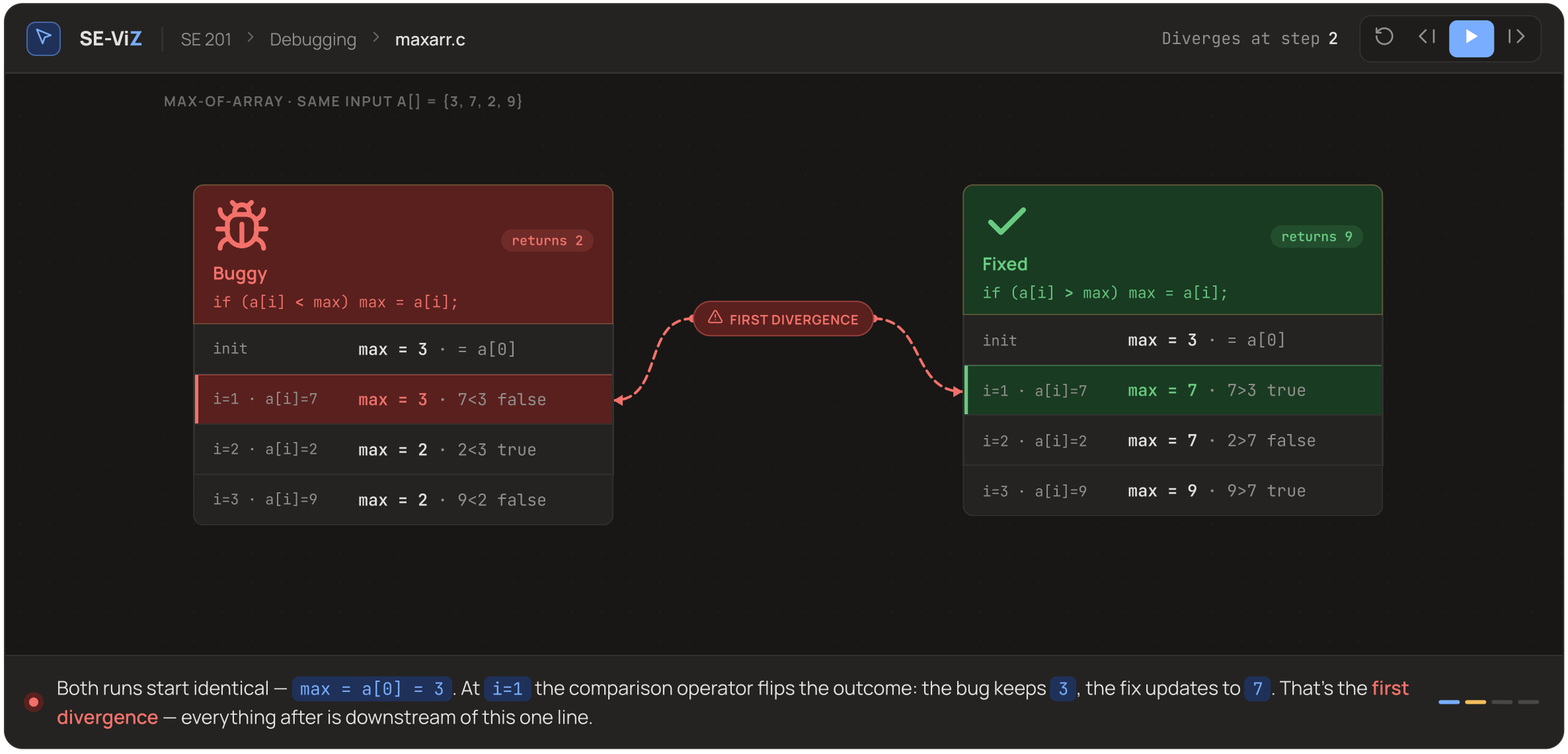
Reading the diagram: flow runs strictly left→right. Dashed neutral boxes (`cc`, `ar`, `ld`) are *processes*; solid cards are *files on disk*. Only the final binary carries an accent — it's the artifact you ship.

08 Preprocessor expansion — `#define` / `#include` substituted textually before compilation



Reading the diagram: the `cpp` process box bridges *before* → *after*. Substitution reuses the *value-changed* green from the memory screens — here it means "this text was transformed," keeping the palette's meaning intact.

09 Debugging — dual trace — same program, buggy vs fixed, with the first point of divergence flagged



Reading the diagram: identical steps render plain; the first row where state differs is flagged red in both columns and linked by the divergence marker. Scanning down to the first red row localizes the bug to a single step — no need to read the whole trace.

Semantic color legend — color is meaning, reused identically on every screen. ~3 hues + state flags, no rainbow coding.

THE MAPPING

Every memory concept owns one consistent color. Learn it once; it holds across recursion, lists, the heap, hash tables and every later screen.

ENTITIES

Pointer · address · arrow Pointer cells, next/prev links, and every connector. The signature accent — azure by default, tweakable.

node

Heap-allocated data Nodes and malloc'd blocks — teal. Carries its own address header.

n

Stack frame · scalar local Neutral / desaturated, so it recedes. The action lives in pointers and heap.

f( )

Process box Hash function, pipeline stage, preprocessor step — dashed neutral, reads as "process," not memory.

STATE FLAGS

ACTIVE

Active / current Amber ring + badge. Exactly one element is current per step.

42

Value just changed A single green pulse that fades — draws the eye to what moved this step.

ALLOCATION STATES (HEAP)

Allocated Normal teal block — live, reachable, owned by a pointer.

Freed Desaturated + hatched. Returned to the allocator, no live references.

⚠

Dangling A pointer still aims at a freed block — flagged red. Reading it is UB.

⚠

Leaked Allocated but unreachable — warning marker, no incoming arrow.

Presentation / fullscreen mode steps with ← / →. Transport controls (reset · step-back · play · step-forward) mirror the keys. This is a static style exploration — the controls show the affordance.